PIXEL

A

MINOR PROJECT REPORT

Submitted in partial fulfillment of the requirements

for the award of the degree of

BACHELOR OF TECHNOLOGY

In

Computer Science & Engineering

Submitted to

RAJIV GANDHI PROUDYOGIKI VISHWAVIDYALAYA,
BHOPAL (M.P.)

Department of Computer Science & Engineering
Sri Aurobindo Institute of Technology, Indore (M.P.)
2025-2026

PIXEL

The Minor Project-II report submitted to

Rajiv Gandhi Proudyogiki Vishwavidyalaya, Bhopal

towards partial fulfillment of the Degree of

Bachelor of Technology

in

Computer Science & Engineering

Guided by
Prof. Kapil Bhardwaj

Submitted by

Jatin Adlak	0873CS231048
Kanishk Kukreja	0873CS231054
Aniket Ambadkar	0873CS231013
Madhav Maurya	0873CS231060
Ansh Jain	0873CS231015
Brajesh Patel	0873CS231027

Department of Computer Science & Engineering
Sri Aurobindo Institute of Technology, Indore(M.P.)

2025-2026

SRI AUROBINDO INSTITUTE OF TECHNOLOGY, INDORE

RECOMMENDATION

This is to certify that Mr. Jatin Adlak (0873CS231048), Mr. Kanishk Kukreja (0873CS231054), Mr. Aniket Ambadkar (0873CS231013), Mr. Madhav Maurya (0873CS231060), Mr. Ansh Jain (0873CS231013) & Mr. Brajesh Patel (0873CS231027) student of B.Tech. VI Semester in the year 2025-26 of Computer Science & Engineering Department of this institute has completed their work on "***PIXEL – Photography Club Website***" for Major Project-II based on syllabus and has submitted a satisfactory account of their work in this report which is recommended for the partial fulfillment of the degree of Bachelor of Technology in Computer Science & Engineering.

Project Guide,
CSE Department
SAIT Indore

HoD,
CSE Department
SAIT Indore

Director,
SAIT Indore

SRI AUROBINDO INSTITUTE OF TECHNOLOGY, INDORE

CERTIFICATE

This is to certify that the work embodied in this project entitled "***PIXEL – Photography Club Website***" being submitted by Mr. Jatin Adlak (0873CS231048), Mr. Kanishk Kukreja (0873CS231054), Mr. Aniket Ambadkar (0873CS231013), Mr. Madhav Maurya (0873CS231060), Mr. Ansh Jain (0873CS231013) & Mr. Brajesh Patel (0873CS231027), student of B.Tech. VI semester, Computer Science & Engineering department in the year 2025-26, is a satisfactory account of their work based on syllabus which is accepted in partial fulfillment of degree of Bachelor of Technology in Computer Science & Engineering.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Date **Date**

SRI AUROBINDO INSTITUTE OF TECHNOLOGY, INDORE

DECLARATION

We hereby declare that the Project entitled **"PIXEL – Photography Club Website"** is our own work conducted under the supervision of **Prof. Kapil Bhardwaj** , Assistant Professor , Department of Computer Science & Engineering at **Sri Aurobindo Institute of Technology, Indore, M.P.**

We further declare that to the best of our knowledge this report does not contain any part of work that has been submitted for the award of any degree either in this institute or in other institute without proper citation.

Jatin Adlak	0873CS231048
Kanishk Kukreja	0873CS231054
Aniket Ambadkar	0873CS231013
Madhav Maurya	0873CS231060
Ansh Jain	0873CS231013
Brajesh Patel	0873CS231027

ACKNOWLEDGEMENT

We express deep gratitude for enthusiasm and valuable suggestions that we got from our guide **Prof. Kapil Bhardwaj** for successful completion of the project. This project was not possible without the invaluable guidance of our project guide.

We are also thankful to our project coordinator **Prof. Sunil Parihar ,** for his technical guidance, encouragement and support.

We are deeply indebted to **Prof. Sunil Parihar**, Head, Department of Computer Science & Engineering, for providing us support and resources for successful completion of this project.

I would like to give thanks to **Dr. Aaquil Bunglowala,** Director SAIT for his valuable guidance and motivation for the completion of the work.

Finally, we are thankful to all the people who are related to the project directly or indirectly.

Jatin Adlak	**0873CS231048**
Kanishk Kukreja	**0873CS231054**
Aniket Ambadkar	**0873CS231013**
Madhav Maurya	**0873CS231060**
Ansh Jain	**0873CS231013**
Brajesh Patel	**0873CS231027**

ABSTRACT

The rapid growth of digital media and event photography has created a demand for efficient systems that can organize, manage, and retrieve images intelligently. This project, titled **"PIXEL – AI Powered Smart Event Gallery and Face Recognition Platform,"** presents the design and implementation of a full-stack web application that automates event photo management and enables users to search for their images using facial recognition technology.

The proposed system integrates modern web technologies with computer vision and artificial intelligence to provide a seamless and interactive user experience. The frontend of the application is developed using React.js to create a responsive and dynamic user interface, while the backend is implemented using Django REST Framework for API management and server-side operations. MongoDB is utilized for efficient storage and management of event data, user details, and image metadata. The project also incorporates FastAPI as a dedicated face recognition microservice to improve processing speed and modularity.

The system uses computer vision and deep learning techniques to detect and recognize faces 11.5 invfrom uploaded event photographs. Face embeddings are generated and compared to identify matching images for users. OpenCV is employed for image processing operations, while machine learning-based face recognition libraries are used to ensure accurate identification and retrieval of photographs.

The platform follows a modular architecture consisting of an event management module, image upload and storage module, AI-based face recognition engine, authentication system, newsletter module, and an interactive gallery interface. Users can browse event galleries, search for their photos using a selfie upload feature, and view newsletters associated with events directly in PDF format. The system also supports real-time image retrieval and optimized API communication between frontend and backend services.The project highlights the practical application of artificial intelligence in media organization, digital event management, and human-centered web platform

TABLE OF CONTENT

Title	Page No.
ABSTRACT	i
TABLE OF CONTENTS	ii
TABLE INDEX	iii
FIGURE INDEX	iv
LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE	v
INTRODUCTION	1
SYSTEM DEVELOPMENT LIFE CYCLE	7
ANALYSIS	13
DESIGN	22
IMPLEMENTATION	31
CONCLUSION	49
REFERENCES	53

LIST OF TABLE

Table No.	Title	Page No.
Table 3.1	Feasibility Analysis Summary	15
Table 5.1	Software Platform Used and their Purpose	33
Table 5.2	Test Cases and their expected Outcomes	47

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 1.1	Conceptual Overview of PIXEL System	6
Figure 2.1	Sequential flow during of SDLC Phases	9
Figure 3.1	Use Case Diagram of PIXEL System	18
Figure 3.2	Activity Diagram of PIXEL System	20
Figure 4.1	System Flow Diagram of PXEL System	24
Figure 4.2	Data Flow Diagram of PIXEL System	25
Figure 4.3	Sequence Diagram for PIXEL System	27
Figure 4.4	Class Diagram for PIXEL System	28
Figure 4.5	ER Diagram for PIXEL System	30
Figure 5.1	User Database Table	36
Figure 5.2	Event Database Table	37
Figure 5.3	Image Database Table	39
Figure 5.4	Post Database Table	40
Figure 5.5	Newsletter Database Table	41
Figure 5.6	Fs chunks Database Table	43

LIST OF SYMBOLS , ABBREVIATIONS AND NOMENCLATURE

Abbreviation / Symbol	Description
AI	Artificial Intelligence
API	Application Programming Interface
UI	User Interface
DB	Database
DFD	Data Flow Diagram
ER Diagram	Entity Relationship Diagram
UML	Unified Modeling Language
SDLC	Software Development Life Cycle
JSON	JavaScript Object Notation
PDF	Portable Document Format
REST	Representational State Transfer
GridFS	MongoDB File Storage System
JWT	JSON Web Token
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
OS	Operating System
FK	Foreign Key
ObjectId	MongoDB Document Identifier

CHAPTERS

CONTENTS	PAGE NO.
CHAPTER 1 : INTRODUCTION	**1**
1.1 Introduction	2
1.2 Problem Statement	3
1.3 Need of the Proposed Project	3
1.4 Objectives of the Project	4
1.5 Scope of the Project	4
1.6 Organization of the Report	5
1.7 Figure Description	6
CHAPTER 2 : SOFTWARE DEVELOPMENT LIFE CYCLE	**7**
2.1 Introduction to Software Development Life Cycle	8
2.2 Phases of Software Development Life Cycle	8
2.3 Requirement Analysis Phase	9
2.4 System Design Phase	9
2.5 Implementation Phase	10
2.6 Testing Phase	10

2.7 Deployment and Maintenance Phase	11
2.8 SDLC Model Adopted	11
2.9 Advantages of Using SDLC in This Project	11
2.10 Summary	12
CHAPTER 3 : SYSTEM ANALYSIS	**13**
3.1 Introduction to System Analysis	14
3.2 Feasibility Study	14
3.2.1 Technical Feasibility	14
3.2.2 Operational Feasibility	15
3.2.3 Economic Feasibility	15
3.3 Requirement Analysis	16
3.3.1 Functional Requirements	16
3.3.2 Non-Functional Requirements	17
3.4 Use Case Analysis	17
3.4.1 Use Case Diagram	18
3.4.2 Explanation of Use Case Diagram	19
3.5 Activity Diagram	20
3.5.1 Explanation of Activity Diagram	21

3.6 Summary	21
CHAPTER 4 : SYSTEM DESIGN	**22**
4.1 Introduction to System Design	23
4.2 System Flow Diagram	23
4.2.1 Definition	23
4.2.2 System Flow Description	23
4.2.3 System Flow Diagram	24
4.3 Data Flow Diagram	25
4.3.1 Definition	25
4.3.2 Data Flow Description	25
4.3.3 Data Flow Diagram	25
4.4 Modules Identified	26
4.4.1 User Management and Authentication Module	26
4.4.2 Event Management Module	26
4.4.3 Image Upload and Processing Module	26
4.4.4 Posts Upload and Processing Module	26
4.4.5 Face Recognition Service Module	26
4.5 Sequence Diagram	27

4.5.1 Definition	27
4.5.2 Sequence Diagram Description	27
4.5.3 Sequence Diagram	27
4.6 Class Diagram	28
4.6.1 Definition	28
4.6.2 Class Diagram	28
4.7 Database Design	29
4.7.1 Definition	29
4.7.2 Database Design Description	29
4.7.3 Entity Relationship Diagram (ER Diagram)	29
4.8 Summary	30
CHAPTER 5 : IMPLEMENTATION	**31**
5.1 Introduction to Implementation Phase	32
5.2 Platform Used	32
5.2.1 Hardware Platform	32
5.2.2 Software Platform	33
5.3 Overall Implementation Architecture	34
5.4 User Management and Authentication Module	34

5.4.1 Database Table Description	35
5.5 Event Management Module	36
5.5.1 Database Table Description	37
5.6 Image Upload and Processing Module	37
5.6.1 Database Table Description	38
5.7 Post Upload and Processing Module	39
5.7.1 Database Table Description	40
5.8 Newsletter Module	40
5.8.1 Database Table Description	41
5.9 Face Recognition and Processing Module	41
5.9.1 Database Table Description	42
5.10 Module Integration	43
5.11 Implementation Summary	44
5.12 Testing	45
5.12.1 Introduction to Testing	45
5.12.2 Testing Objectives	45
5.12.3 Testing Techniques Used	45
5.12.4 Test Cases Used During Implementation	46

5.12.5 Test Environment	47
5.12.6 Testing Limitations	47
5.12.7 Testing Summary	48
CHAPTER 6 : CONCLUSION	**49**
6.1 Introduction	50
6.2 Important Features of the System	50
6.3 Limitations of the System	51
6.4 Future Work	51
6.5 Summary	52
REFERENCES	**53**

CHAPTER 1

INTRODUCTION

1.1 Introduction

In the modern digital era, photography has become an essential part of social, academic, and professional events. Large numbers of photographs are captured during college fests, workshops, seminars, cultural programs, sports events, and other gatherings. Managing and organizing these images efficiently has become a major challenge due to the continuously increasing volume of digital media. Traditional event gallery systems mainly rely on manual categorization and browsing, making it difficult for users to locate their personal photographs quickly and accurately. This creates a need for an intelligent and automated solution capable of simplifying event photo management and retrieval.

The project **"PIXEL – AI Powered Smart Event Gallery and Face Recognition Platform"** is designed to address these challenges by integrating modern web technologies with artificial intelligence and computer vision techniques. The platform provides a centralized digital environment where event photographs can be uploaded, managed, processed, and searched efficiently. One of the key features of the system is AI-based face recognition, which enables users to upload a selfie and automatically retrieve matching photographs from event galleries without manually browsing through hundreds of images.

The system is developed using a modern full-stack architecture. The frontend is implemented using React.js to provide an interactive and responsive user interface, while the backend services are developed using Django REST Framework for API handling and FastAPI for high-performance face recognition processing. MongoDB is used as the database system for storing user information, event details, image metadata, and face embeddings. OpenCV and machine learning-based face recognition techniques are utilized for image processing and intelligent face matching.

The platform consists of multiple integrated modules such as user authentication, event management, image upload and gallery handling, AI-based face search, newsletter management, and dataset processing services. The architecture follows a modular and scalable design, ensuring efficient communication between frontend and backend components. Automated image indexing and dataset monitoring mechanisms improve the efficiency and responsiveness of the system.

The proposed solution not only enhances user experience but also demonstrates the practical application of artificial intelligence in media management and event organization systems. The project provides a fast, reliable, and scalable platform for smart image retrieval and digital event management. It highlights the growing importance of AI-driven applications in improving automation, accessibility, and efficiency in modern web-based systems.

1.2 Problem Statement

Managing and searching photographs from large events such as college fests, workshops, and cultural programs is difficult and time-consuming using traditional gallery systems. Users often need to manually browse through hundreds of images to find their photographs. Existing systems lack intelligent face-based image retrieval and efficient event photo management. Therefore, there is a need for a smart platform that uses artificial intelligence and face recognition to organize event galleries and help users quickly search and retrieve their photos efficiently.

1.3 Need of the Proposed Project

In colleges and educational institutions, a large number of photographs are captured during technical events, cultural fests, workshops, seminars, sports activities, and social gatherings. Managing these images manually becomes difficult as users often need to scroll through hundreds of photos to find their own pictures. Traditional gallery systems do not provide intelligent search features, making the process time-consuming and inefficient. Therefore, there is a need for a smart event management and image retrieval platform that can organize event photographs efficiently and allow users to search their images quickly using AI-based face recognition technology.

The project also provides a social and interactive platform for students by allowing them to browse different college events, explore student profiles, connect with classmates and other students, and stay updated with campus activities. Along with smart photo searching, the platform creates a centralized digital community where users can share memories, discover event highlights, and interact with others through a modern and user-friendly interface.

1.4 Objectives of the Project

The main objectives of the **PIXEL - Photography Club Website** project are:

- To develop an AI-powered smart event gallery platform for managing and organizing college event photographs efficiently.
- To implement face recognition technology for fast and accurate photo searching and retrieval.
- To reduce the time and effort required to manually browse large image collections.
- To provide a centralized platform for browsing college events, galleries, and newsletters.
- To enable students to explore profiles and connect with classmates and other students.
- To build a responsive and user-friendly full-stack web application using modern technologies.
- To improve digital event management through automation and intelligent image processing.
- To create a scalable system capable of handling large amounts of event data and media files..

1.5 Scope of the Project

The scope of the project includes the development of an AI-powered smart event gallery platform for colleges and educational institutions. The system allows users to upload, manage, browse, and search event photographs efficiently through a centralized digital platform. It supports multiple college events such as technical fests, workshops, cultural programs, seminars, and sports activities.

The project also includes the implementation of face recognition technology for intelligent image retrieval, enabling users to quickly find their photographs using a selfie upload feature. In addition, the platform provides social interaction features where students can browse event galleries, view newsletters, explore profiles, and connect with classmates and other students. The system is designed to be scalable, user-friendly, and suitable for future enhancements such as real-time notifications, mobile applications, cloud deployment, and advanced AI-based analytics.

1.6 Organization of the Report

This report is organized into different chapters describing the development and implementation of the "PIXEL – Smart Event Gallery and Student Interaction Platform."

Chapter 1: Introduction

This chapter provides the overview of the project, including the problem statement, need of the project, objectives, scope, and introduction to the proposed system.

Chapter 2: System Development Life Cycle

This chapter explains the development methodology and different phases followed during the project development process.

Chapter 3: Analysis

This chapter describes the requirement analysis, system requirements, and overall analysis of the proposed platform along with use case and activity analysis.

Chapter 4: Design

This chapter presents the system architecture, data flow diagrams, database design, module identification, and other design-related components of the project.

Chapter 5: Implementation

This chapter explains the technologies used, frontend and backend implementation, AI-based image search integration, database connectivity, and testing procedures.

Chapter 6: Conclusion

This chapter summarizes the outcomes of the project, important features, limitations, and future scope of the system.

Chapter 7: References

This chapter contains the references of technologies, documentation, research papers, and online resources used during the development of the project.

1.7 Figure Description

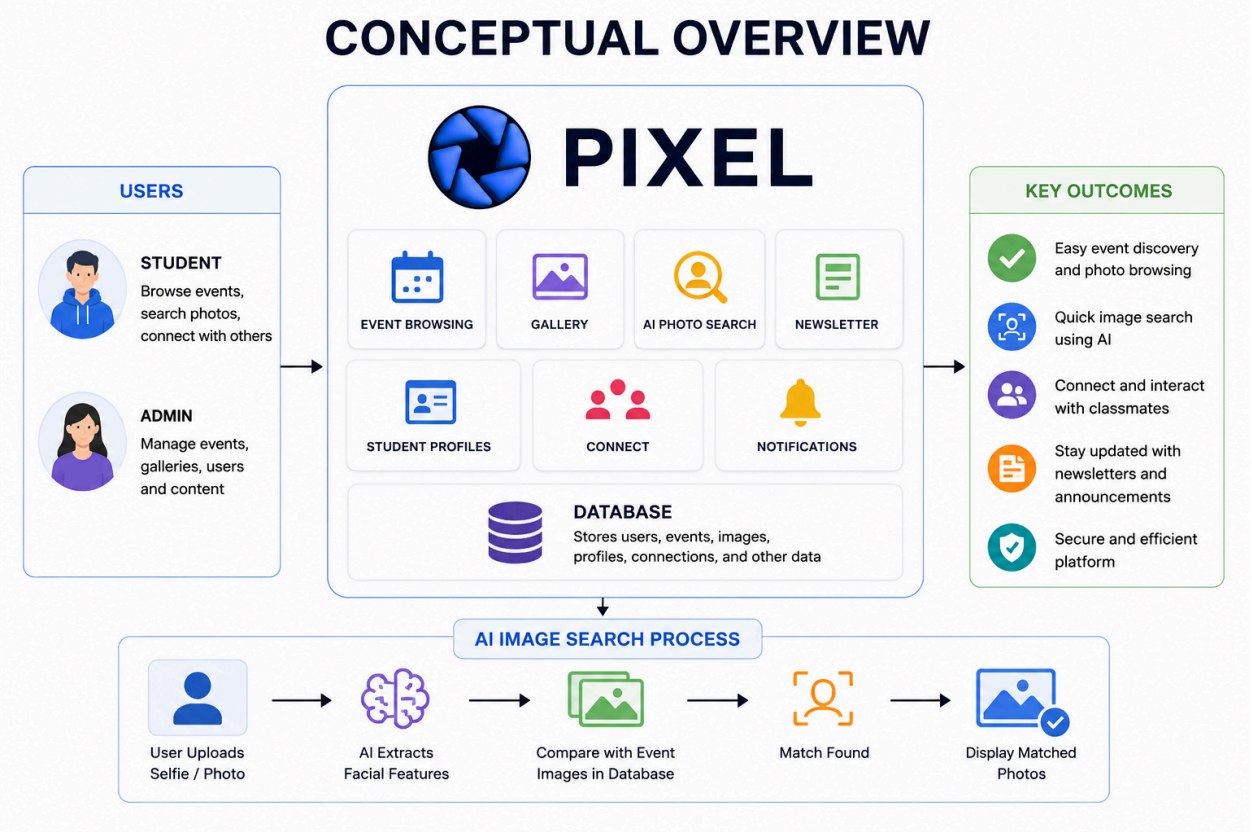

Figure 1.1 The conceptual overview diagram represents the architecture and workflow of the PIXEL platform, including event browsing, AI-based image search, student interaction, and centralized data management.

CHAPTER 2

SOFTWARE DEVELOPMENT LIFE CYCLE

2.1 Introduction to Software Development Life Cycle

The development of the PIXEL platform followed an incremental and modular Software Development Life Cycle (SDLC) approach, where different components of the system were developed and integrated step-by-step. The project development started with the design and implementation of the frontend interface using React.js to create a responsive and interactive user experience for browsing events, galleries, profiles, and other platform features.

After completing the frontend structure, a separate AI-based image search service was developed using FastAPI, OpenCV, and face recognition techniques. This module was designed independently to handle image processing, facial feature extraction, and photo matching functionalities efficiently. The modular approach helped in isolating the AI processing logic from the main application, improving scalability and maintainability.

In the next phase, the backend services and database integration were implemented using Django REST Framework and MongoDB. APIs were developed for user authentication, event management, image handling, newsletter management, and communication between frontend and AI services. Database schemas and storage mechanisms were also designed to manage users, events, images, and metadata efficiently.

Finally, all components including the frontend, backend, database, and AI image search service were integrated into a complete full-stack platform. Extensive testing and debugging were performed to ensure proper communication between modules, smooth user interaction, accurate image retrieval, and overall system stability.

2.2 Phases of Software Development Life Cycle

The Software Development Life Cycle consists of several sequential phases, each focusing on a specific aspect of software development. The major phases followed in this project are:

1. Planning Phase
2. Requirement Analysis Phase
3. System Design Phase
4. Development Phase
5. Testing Phase
6. Deployment and Maintenance Phase

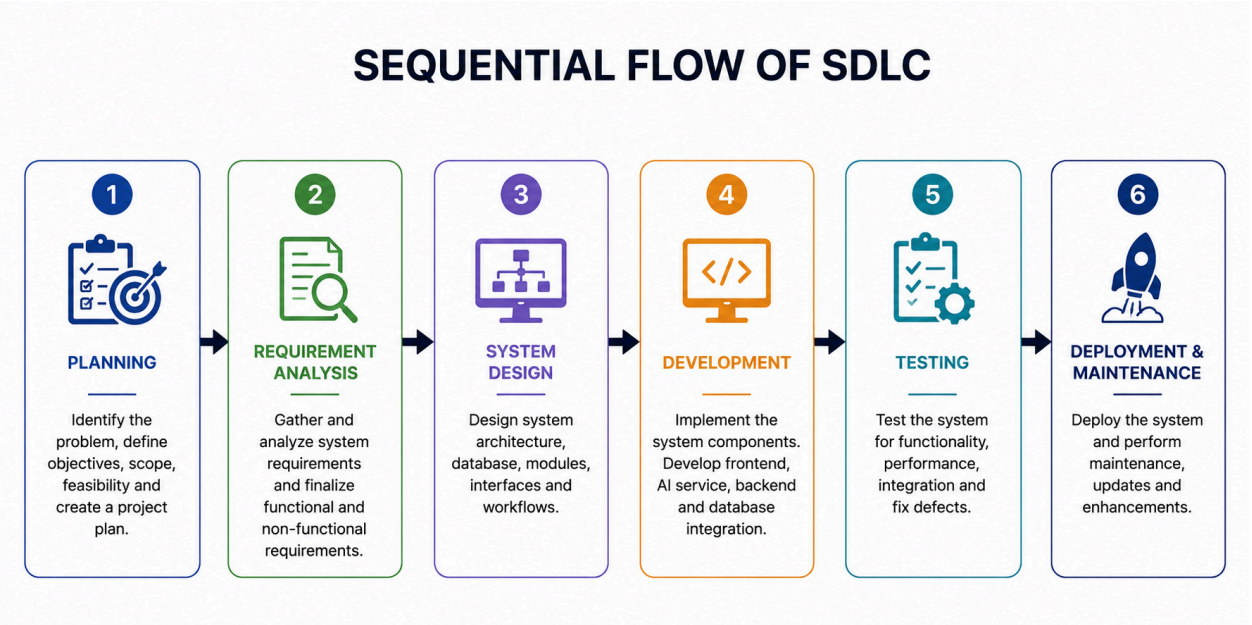

Figure 2.1 Sequential flow of the Software Development Life Cycle (SDLC) followed during the development of the PIXEL platform.

2.3 Requirement Analysis Phase

In the Requirement Analysis phase, the requirements and functionalities of the PIXEL platform were identified and analyzed. The main objective was to understand the problems faced in traditional event gallery systems, where users need to manually browse large numbers of images to find their photographs. The need for an intelligent platform with AI-based image search, event browsing, and student interaction features was studied.

Based on the analysis, functional requirements such as user authentication, event management, image uploads, gallery browsing, AI-based photo search, student profiles, and newsletter viewing were finalized. Non-functional requirements including performance, scalability, responsiveness, and security were also identified. The technologies required for frontend development, backend APIs, database management, and AI image processing were selected during this phase.

2.4 System Design Phase

In the System Design phase, the overall architecture and structure of the PIXEL platform were planned and designed. The system modules, user interface flow, database structure, API

communication, and AI image search workflow were defined to ensure smooth interaction between all components of the platform.

The frontend design was planned using React.js to provide a responsive and user-friendly interface for event browsing, galleries, profiles, and image searching. The backend architecture was designed using Django REST Framework for handling APIs and MongoDB for storing user data, events, images, and metadata. A separate AI image search service was designed using FastAPI and computer vision techniques to process uploaded images and retrieve matching photographs efficiently.

During this phase, system flow diagrams, data flow diagrams, database schemas, and module interactions were also prepared to provide a clear blueprint for the development process.

2.5 Implementation Phase

In the Implementation phase, the planned design of the PIXEL platform was converted into a working system using modern web technologies and AI-based image processing techniques. The development process started with the frontend implementation using React.js to create responsive user interfaces for event browsing, galleries, profiles, newsletters, and image search features.

After frontend development, a separate AI image search service was implemented using FastAPI, OpenCV, and face recognition techniques for image processing and matching functionalities. The backend APIs and database integration were then developed using Django REST Framework and MongoDB to manage users, events, images, authentication, and communication between all modules.

Finally, all system components including the frontend, backend, database, and AI service were integrated to form a complete full-stack platform.

2.6 Testing Phase

In the Testing phase, the complete PIXEL platform was tested to verify the functionality, performance, and reliability of all system modules. Different features such as user authentication, event browsing, image uploads, AI-based image search, profile management, and newsletter viewing were tested under different conditions.

API communication between the frontend, backend, database, and AI image search service was also tested to ensure smooth data transfer and proper integration. Errors related to image retrieval, system performance, and module interaction were identified and resolved to improve the stability and accuracy of the platform.

2.7 Deployment and Maintenance Phase

In the Deployment and Maintenance phase, all modules of the PIXEL platform including the frontend, backend, database, and AI image search service were integrated and prepared for execution as a complete system. The platform was configured to ensure smooth communication between different services and efficient system performance.

Maintenance activities include fixing bugs, improving system performance, updating features, and enhancing the overall user experience. Future improvements and scalability options can also be implemented during this phase to support additional functionalities and advanced features.

2.8 SDLC Model Adopted

The Incremental SDLC Model was adopted for the development of the PIXEL platform. In this model, the project was developed in multiple phases by implementing and integrating different modules step-by-step. This approach helped in developing the system efficiently while allowing continuous improvements during the development process.

The frontend interface was developed first, followed by the AI image search service, backend APIs, and database integration. After developing individual modules, all components were integrated and tested together to form the complete system. The incremental model provided flexibility, easier debugging, better module management, and smooth integration of advanced features into the platform.

2.9 Advantages of Using SDLC in This Project

- Helped in organizing the development process in a systematic and structured manner.
- Made it easier to develop different modules such as frontend, backend, database, and AI image search separately.
- Improved project management and reduced development complexity.

- Allowed continuous testing and debugging during each phase of development.
- Simplified the integration of multiple technologies and services.
- Enhanced system reliability, scalability, and maintainability.
- Helped in identifying and resolving errors at early stages of development.
- Provided flexibility for adding future enhancements and new features to the platform.

2.10 Summary

The Software Development Life Cycle (SDLC) provided a structured and systematic approach for the development of the PIXEL platform. Different phases such as planning, requirement analysis, design, development, testing, and deployment helped in organizing the project efficiently. The use of the Incremental SDLC Model allowed separate development and smooth integration of the frontend, backend, database, and AI image search service. This approach improved system reliability, scalability, maintainability, and overall development efficiency.

CHAPTER 3

SYSTEM ANALYSIS

3.1 Introduction to System Analysis

System Analysis is an important phase in the development of the **PIXEL** platform, where the existing problems, system requirements, and functionalities were carefully studied and evaluated. This phase focuses on understanding user needs, identifying limitations of traditional event gallery systems, and analyzing the requirements for building an intelligent and efficient platform for event management and image retrieval.

The analysis process helped in defining the functional and non-functional requirements of the system, including event browsing, image uploads, AI-based image search, student interaction, profile management, and newsletter access. It also helped in selecting suitable technologies and designing a scalable architecture capable of handling frontend operations, backend services, database management, and AI-based processing efficiently.

This phase provided a clear understanding of the overall system workflow and established the foundation for the design and implementation of the **PIXEL** platform.

3.2 Feasibility Study

A feasibility study evaluates whether the proposed system is practical and viable. The feasibility of the PIXEL system is analyzed under the following categories.

3.2.1 Technical Feasibility

The PIXEL platform is technically feasible because the required technologies, tools, and development resources are readily available and suitable for implementing the system. Modern frameworks such as React.js, Django REST Framework, FastAPI, and MongoDB provide efficient support for developing scalable full-stack web applications.

The AI-based image search functionality can be implemented effectively using computer vision and face recognition libraries such as OpenCV. The system architecture supports smooth integration between frontend, backend, database, and AI services, ensuring reliable performance and efficient communication between modules.

3.2.2 Operational Feasibility

The PIXEL platform is operationally feasible because it is designed with a simple, interactive, and user-friendly interface that can be easily used by students and administrators. Users can browse events, search photographs, view profiles, and access newsletters without requiring technical expertise.

The system automates image management and retrieval processes, reducing manual effort and improving overall efficiency. Its modular structure and responsive design ensure smooth operation and easy maintenance of the platform.

3.2.3 Economic Feasibility

The PIXEL platform is economically feasible because it is developed mainly using open-source technologies such as React.js, Django REST Framework, FastAPI, MongoDB, and OpenCV, which reduces development and software costs. The project does not require expensive hardware or licensed software for implementation.

The system provides a cost-effective solution for managing event galleries and AI-based image search while offering scalability and flexibility for future enhancements and feature additions.

Feasibility Type	Description
Technical Feasibility	Uses open-source libraries and standard hardware
Operational Feasibility	Easy to use and user-friendly
Economic Feasibility	No additional cost involved

Table 3.1 Feasibility Analysis Summary

3.3 Requirement Analysis

Requirement Analysis is the process of identifying, studying, and defining the functional and non-functional requirements of the PIXEL platform. In this phase, the needs of users and the limitations of traditional event gallery systems were analyzed to design an efficient and intelligent solution.

The analysis focused on features such as event browsing, image uploads, AI-based image search, student profiles, newsletters, and user interaction. Functional requirements related to system operations and non-functional requirements such as performance, scalability, security, and responsiveness were also identified during this phase.

This phase helped in selecting suitable technologies, defining system functionalities, and creating a clear roadmap for the design and implementation of the platform.

3.3.1 Functional Requirements

Functional requirements specify the **core operations and services** provided by the system. The following are the functional requirements of the PIXEL system:

- Requirement Analysis is the process of identifying, studying, and defining the functional and non-functional requirements of the PIXEL platform. In this phase, the needs of users and the limitations of traditional event gallery systems were analyzed to design an efficient and intelligent solution.
- The analysis focused on features such as event browsing, image uploads, AI-based image search, student profiles, newsletters, and user interaction. Functional requirements related to system operations and non-functional requirements such as performance, scalability, security, and responsiveness were also identified during this phase.
- This phase helped in selecting suitable technologies, defining system functionalities, and creating a clear roadmap for the design and implementation of the platform.
- The system should allow users to access event details, announcements, and updates through the platform.
- The platform should provide responsive user interfaces for desktop and mobile devices.

3.3.2 Non-Functional Requirements

Non-functional requirements define the **quality attributes** and constraints of the system. The non-functional requirements of the PIXEL system are as follows:

- **Performance:** The system shall provide fast event browsing, image retrieval, and API response with minimal delay.
- **Accuracy:** The system shall provide reliable AI-based image search and matching results.
- **Usability:** The platform shall provide an interactive, responsive, and easy-to-use user interface.
- **Reliability:** The system shall ensure stable communication between frontend, backend, database, and AI services.
- **Scalability:** The platform shall support increasing numbers of users, events, and image data in the future.
- **Security:** The system shall provide secure authentication and protect user and event data.
- **Portability:** The platform shall operate on different browsers and devices with minimal configuration.
- **Maintainability:** The system shall follow a modular architecture for easier debugging, updates, and future enhancements.

3.4 Use Case Analysis

Use Case Analysis is used to identify the interactions between users and the PIXEL platform. It describes how different users utilize the system functionalities to perform various operations such as browsing events, uploading images, searching photographs, and managing event data.

The main actors of the system are Users and Administrators. Users can create accounts, browse events, view galleries, search photographs using AI-based image search, access newsletters, explore student profiles, and connect with other students. Administrators can manage events, upload images, manage newsletters, and monitor platform activities. Use Case Analysis helps in understanding the overall workflow of the system.

3.4.1 Use Case Diagram

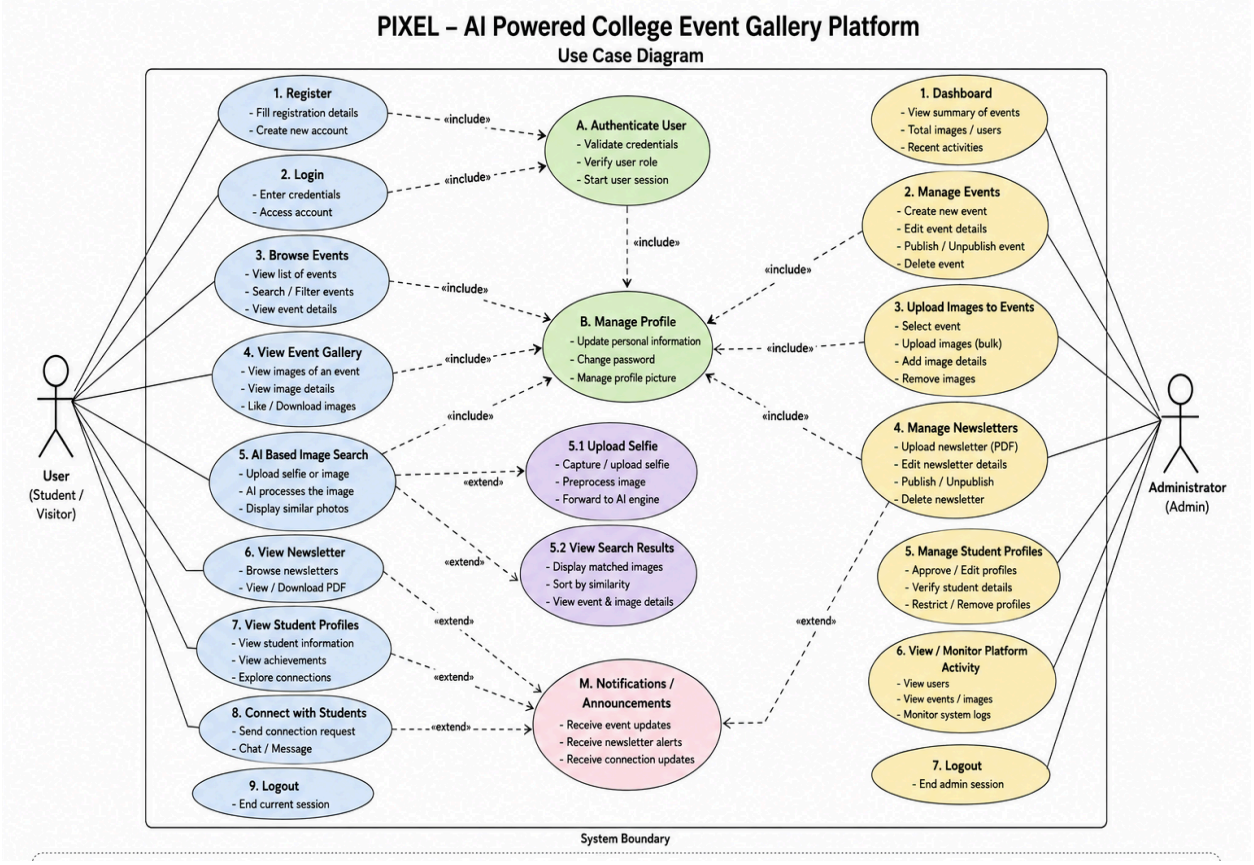

Figure 3.1 shows the Use Case Diagram of the PIXEL system.

Actors:

- **User (Student / Visitor)**

  A user can register, login, browse events, view galleries, search photos, access newsletters, explore student profiles, and connect with other students.

- **Administrator (Admin)**

  The administrator manages events, uploads images, manages newsletters, verifies student profiles, monitors platform activities, and controls overall system operations.

Use Cases:

1. **User Use Cases -**
    - Register
    - Login
    - Browse Events
    - View Event Gallery
    - AI-Based Image Search
    - Upload Selfie for Search
    - View Search Results
    - View Newsletter
    - View Student Profiles
    - Connect with Students
    - Receive Notifications and Announcements
    - Logout

2. **Administrator Use Cases -**
    - Login
    - Manage Events
    - Upload Images to Events
    - Manage Newsletters
    - Manage Student Profiles
    - View / Monitor Platform Activity
    - Send Notifications and Announcements
    - Logout

Explanation of Use Case Diagram

The Use Case Diagram represents the interactions between users, administrators, and the PIXEL platform. It illustrates the main functionalities of the system such as event browsing, image search, profile management, newsletter handling, and platform administration. The diagram helps in understanding the overall workflow and user activities within the system.

3.5 Activity Diagram

The activity diagram represents the **workflow of operations** performed by the system from start to finish. It helps in understanding the sequence of activities within the system.

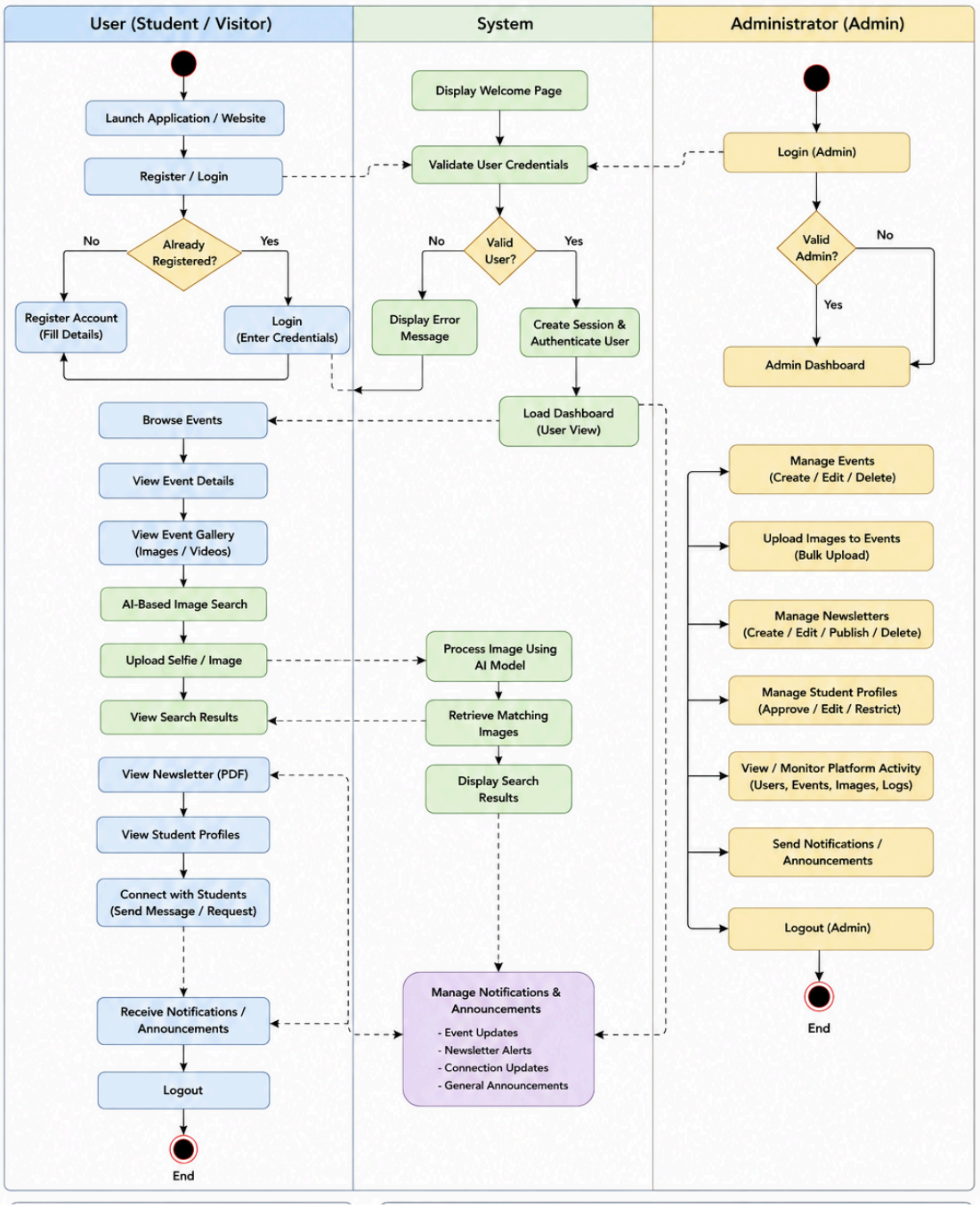

Figure 3.2 Activity Diagram of PIXEL System

Activity Flow:

1. User launches the PIXEL platform and registers or logs into the system.
2. The system validates user credentials and provides access to the dashboard.
3. Users can browse events, view galleries, access newsletters, explore student profiles, and connect with other students.
4. For AI-based image search, the user uploads a selfie or image to the platform.
5. The system processes the uploaded image using the AI image search service and retrieves matching photographs from the database.
6. The matched images are displayed to the user as search results.
7. Administrators can manage events, upload images, manage newsletters, verify student profiles, and monitor platform activities.
8. The system sends notifications and announcements related to events, newsletters, and user activities.
9. Finally, users and administrators can securely logout from the platform.

Explanation of Activity Diagram

The Activity Diagram represents the workflow and sequence of activities performed within the PIXEL platform. It illustrates how users and administrators interact with the system for functionalities such as authentication, event browsing, AI-based image search, profile management, notifications, and platform administration. The diagram helps in understanding the overall flow of operations and system behavior.

3.6 Summary

The System Analysis chapter provided a detailed understanding of the requirements, feasibility, and overall workflow of the PIXEL platform. It analyzed the functional and non-functional requirements of the system and evaluated the technical, operational, and economic feasibility of the proposed solution.The chapter also explained the interactions between users, administrators, and the platform through use case and activity analysis. These analyses helped in understanding the system behavior, user activities, and module interactions, which formed the foundation for the system design and implementation phases.

CHAPTER 4

SYSTEM DESIGN

4.1 Introduction to System Design

System design is a crucial phase in the software development process where the conceptual requirements of a system are transformed into a structured and detailed technical blueprint. This phase defines the architecture, components, data flow, and interaction between different modules of the system.

For the FRACTAL – Gesture Controlled OS Interface, system design focuses on integrating computer vision, gesture recognition, operating system automation, and a web-based user interface into a unified system. The design ensures that the system performs efficiently in real time while remaining modular, scalable, and easy to maintain.

4.2 System Flow Diagram

4.2.1 Definition

A System Flow Diagram is a graphical representation that illustrates the overall flow of data, processes, and interactions within a system. It helps in understanding how different modules and components communicate with each other to perform system operations efficiently.

4.2.2 System Flow Description

The System Flow Diagram of the PIXEL platform represents the interaction between users, administrators, system processes, and the database. Users can register, browse events, search images, view newsletters, explore profiles, and connect with other students through the platform. The system processes user requests, performs AI-based image search operations, retrieves data from the database, and displays results to users.

Administrators manage events, images, newsletters, student profiles, notifications, and platform activities. The database stores information related to users, events, galleries, newsletters, notifications, and system logs. The diagram provides a clear overview of the workflow and communication between all major components of the PIXEL platform.

4.2.3 System Flow Diagram

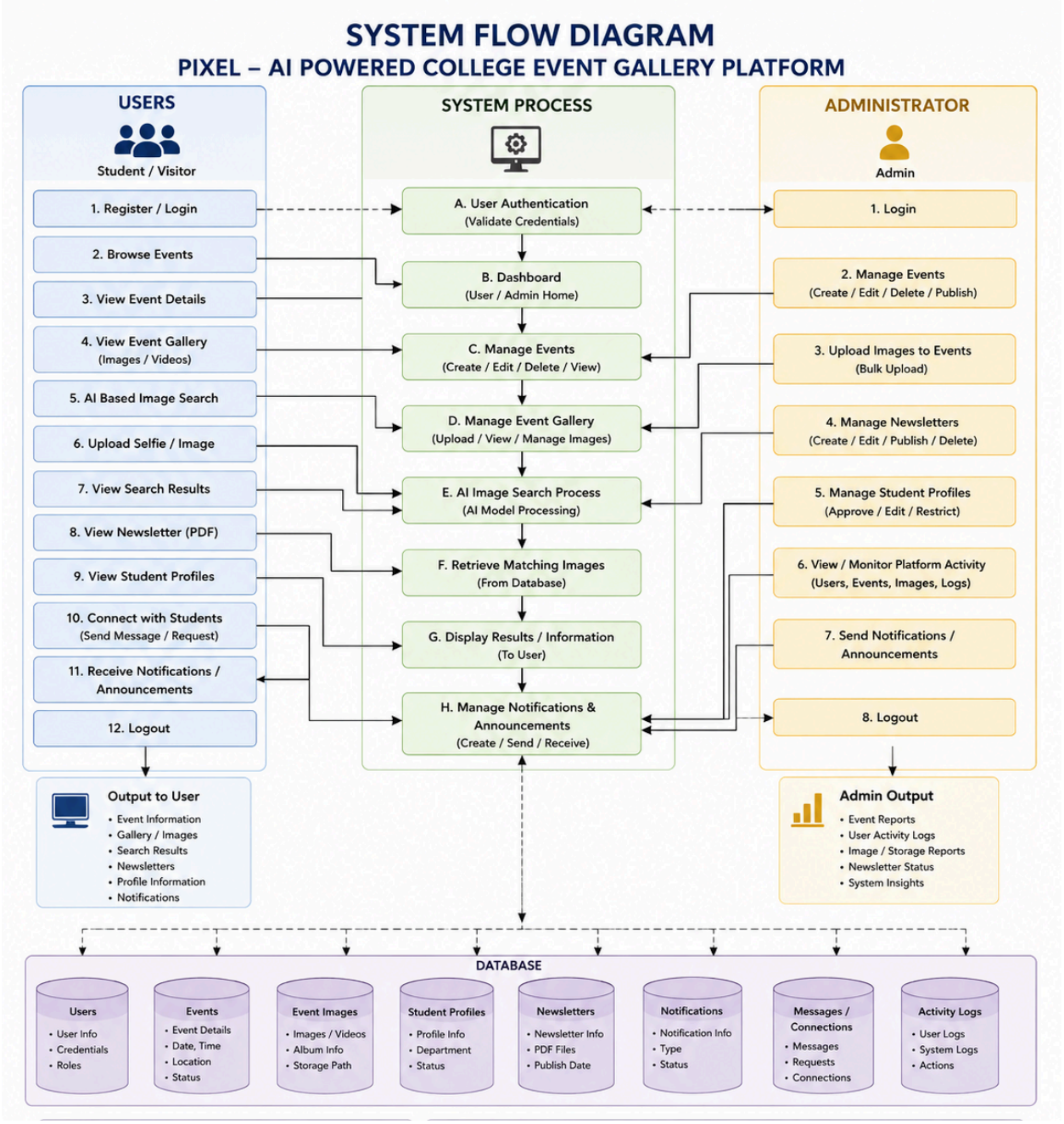

Figure 4.1: System Flow Diagram of PIXEL System

4.3 Data Flow Diagram

4.3.1 Definition

A Data Flow Diagram (DFD) is a graphical representation that illustrates how data moves through a system. It shows the flow of information between external entities, system processes, and data storage components.

4.3.2 Data Flow Description

The Data Flow Diagram of the PIXEL platform represents the flow of data between users, administrators, the system, and the database. Users perform actions such as login, event browsing, image search, and profile viewing, while administrators manage events, images, newsletters, and platform activities. The system processes these requests, stores and retrieves data from the database, and returns the required information and results to users.

4.3.3 Data Flow Diagram

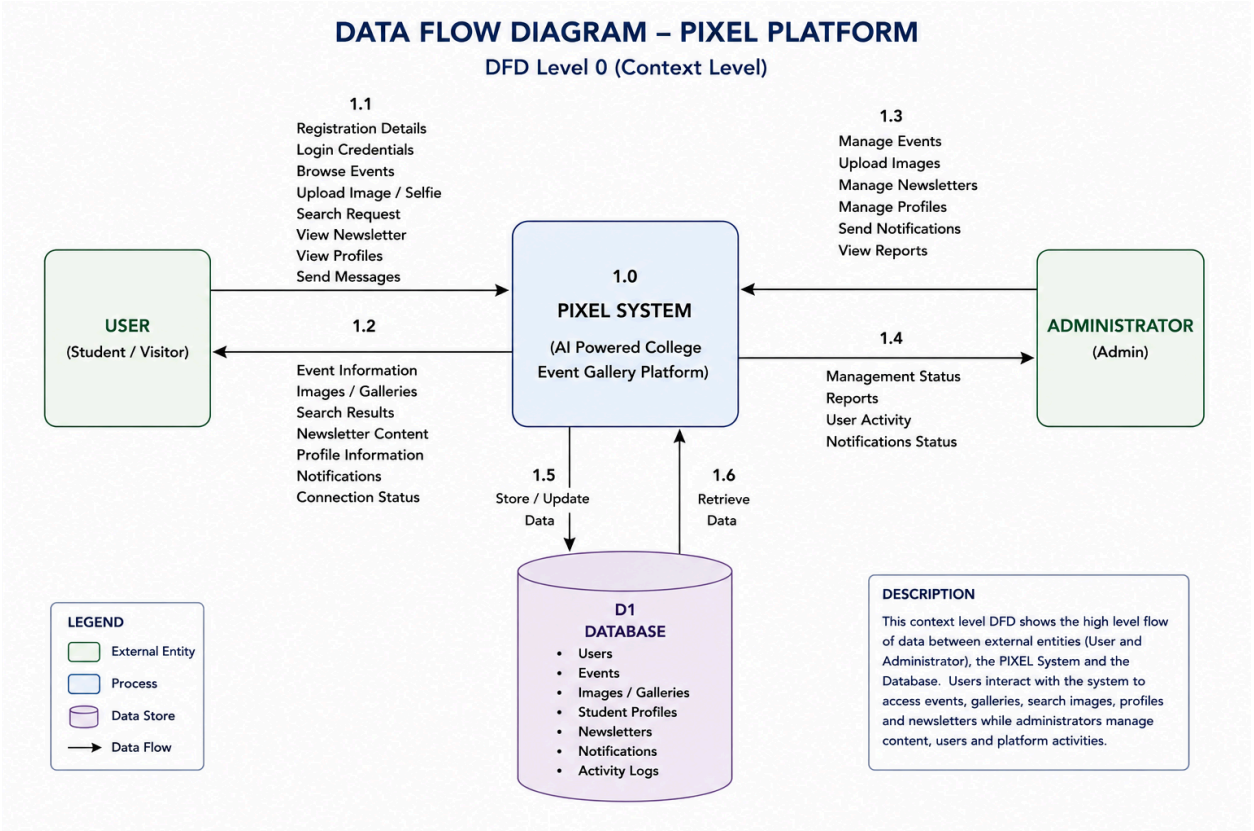

Figure 4.2: Data Flow Diagram of PIXEL System

4.4 Modules Identified

The PIXEL system follows a modular architecture to ensure separation of concerns and ease of future enhancement. The major modules identified are as follows:

4.4.1 User Management and Authentication Module

This module manages user registration, login, authentication, and profile handling within the PIXEL platform. It ensures secure access to the system and allows users to create accounts, manage profiles, and interact with different platform features securely.

4.4.2 Event Management Module

The Event Management Module allows administrators to create, update, and manage different college events such as workshops, fests, seminars, and cultural activities. It helps organize event details, galleries, announcements, and related information within the platform.

4.4.3 Image Upload and Processing Module

This module handles the uploading, storage, and processing of event images. It manages image organization, retrieval, and communication with the database and AI services to ensure efficient image handling within the platform.

4.4.4 Posts Upload and Processing Module

The Posts Upload and Processing Module allows users and administrators to upload and manage posts related to events, announcements, and campus activities. It helps users stay updated with important information and interact with platform content.

4.4.5 Face Recognition Service Module

This module is responsible for AI-based image search and face matching functionalities. It processes uploaded images using computer vision techniques, compares facial features with stored event images, and retrieves matching photographs efficiently.

4.5 Sequence Diagram

4.5.1 Definition

A Sequence Diagram is a type of UML diagram that represents the sequence of interactions and communication between different system components over time. It illustrates how objects and modules interact with each other to perform a specific operation within the system.

4.5.2 Sequence Diagram Description

The Sequence Diagram of the PIXEL platform represents the interaction between users, frontend, backend services, database, and the AI-based image search service. It shows the sequence of operations such as user authentication, event browsing, image uploads, post handling, newsletter access, and AI-based image search. The diagram helps in understanding the communication flow and execution order between different modules of the system.

4.5.3 Sequence Diagram

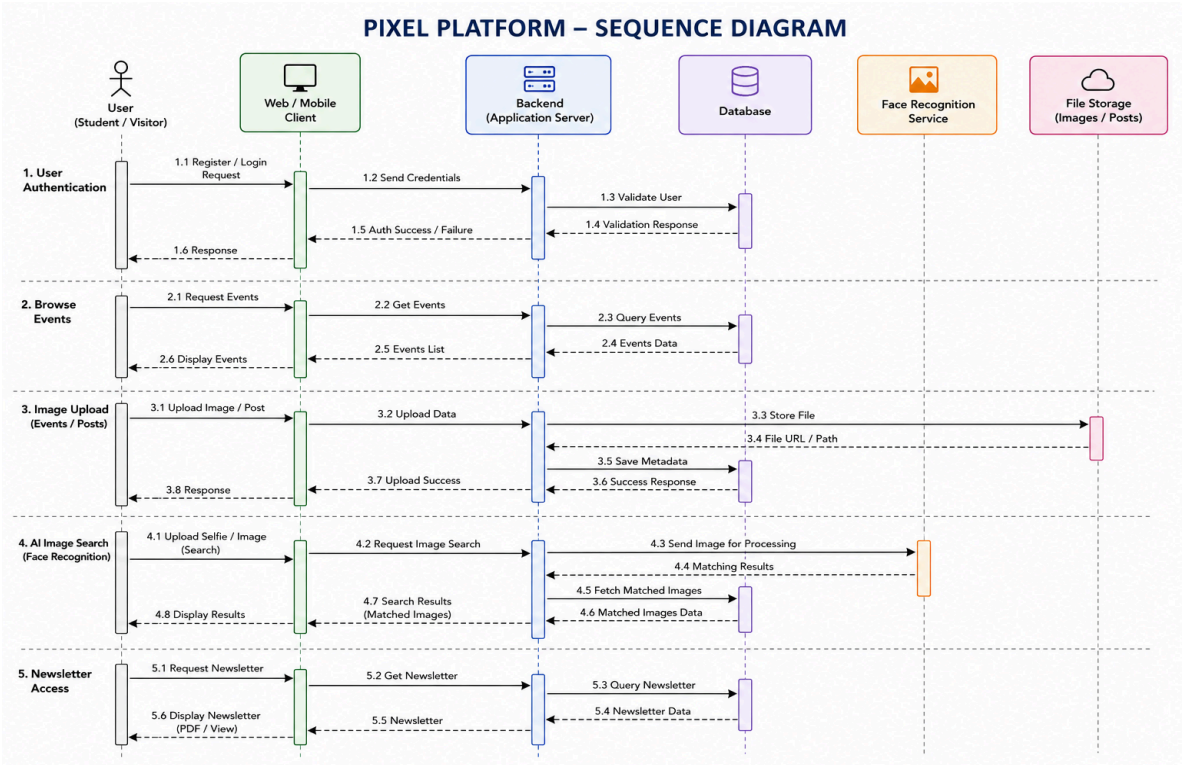

Figure 4.3: Sequence Diagram for Pixel System

4.6 Class Diagram

4.6.1 Definition

A Sequence Diagram is a type of UML diagram that represents the sequence of interactions and communication between different system components over time. It illustrates how objects and modules interact with each other to perform a specific operation within the system.

4.6.2 Class Diagram

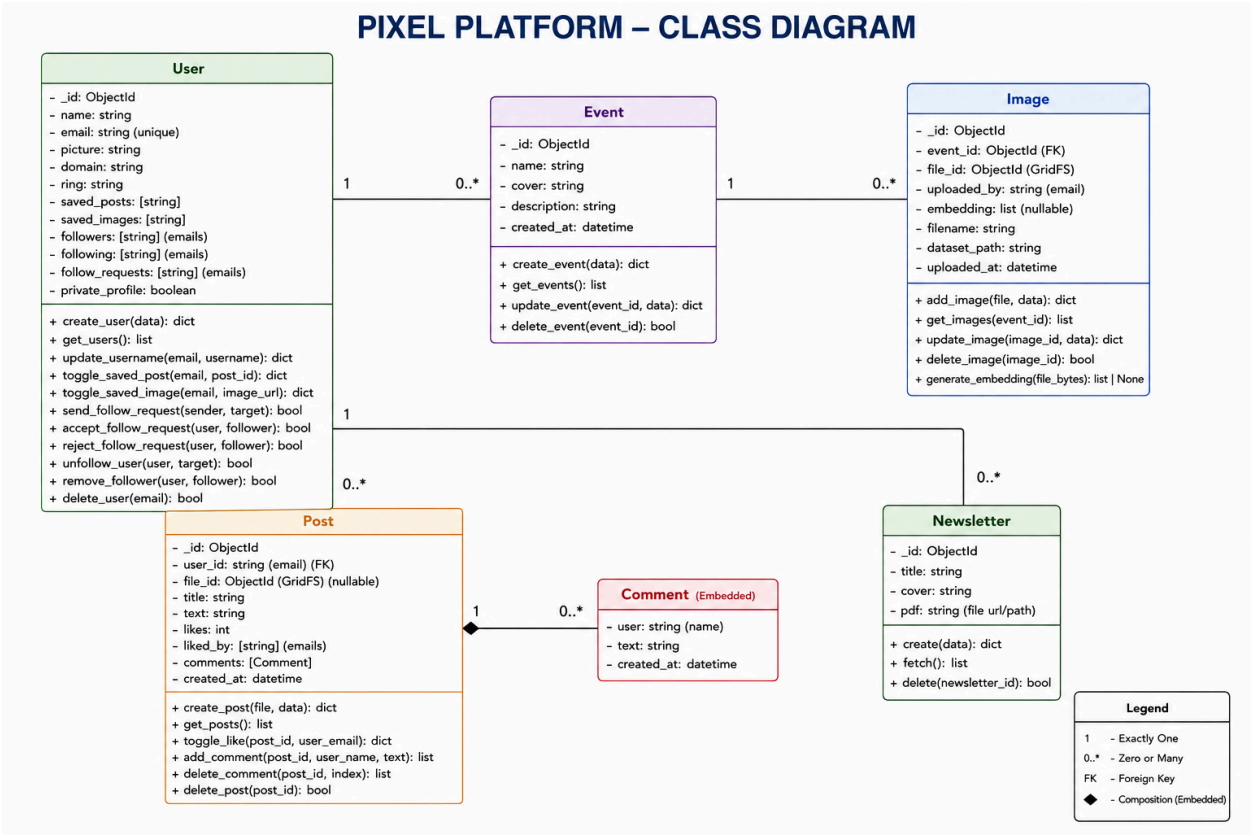

Figure 4.4: Class Diagram for Pixel System

4.7 Database Design

4.7.1 Definition

Database Design is the process of organizing, structuring, and managing data within a database system. It defines how data is stored, related, retrieved, and maintained efficiently for system operations.

4.7.2 Database Design Description

The PIXEL platform uses MongoDB as its primary database for storing and managing application data. The database is designed to handle users, events, images, posts, newsletters, comments, and social interactions efficiently. Collections are used to store structured data related to different modules of the system.

The database design supports fast data retrieval, image management, AI-based image search operations, and smooth communication between frontend, backend, and storage services. GridFS is used for storing large image files and media content, while metadata and user information are maintained in separate collections for better scalability and performance.

4.7.3 Entity Relationship Diagram (ER Diagram)

The Entity Relationship (E-R) Diagram of the PIXEL platform represents the relationships between different entities such as Users, Events, Images, Posts, Comments, and Newsletters. It illustrates how data is organized and connected within the database system.

A User can create posts, upload images, participate in events, and interact with other users through follow requests and social connections. Events contain multiple images and related information, while posts can contain comments and user interactions such as likes. The Newsletter entity stores digital newsletters related to college activities and events.

The E-R diagram helps in understanding the database structure, entity relationships, and data flow within the PIXEL platform, ensuring efficient storage and management of system data.

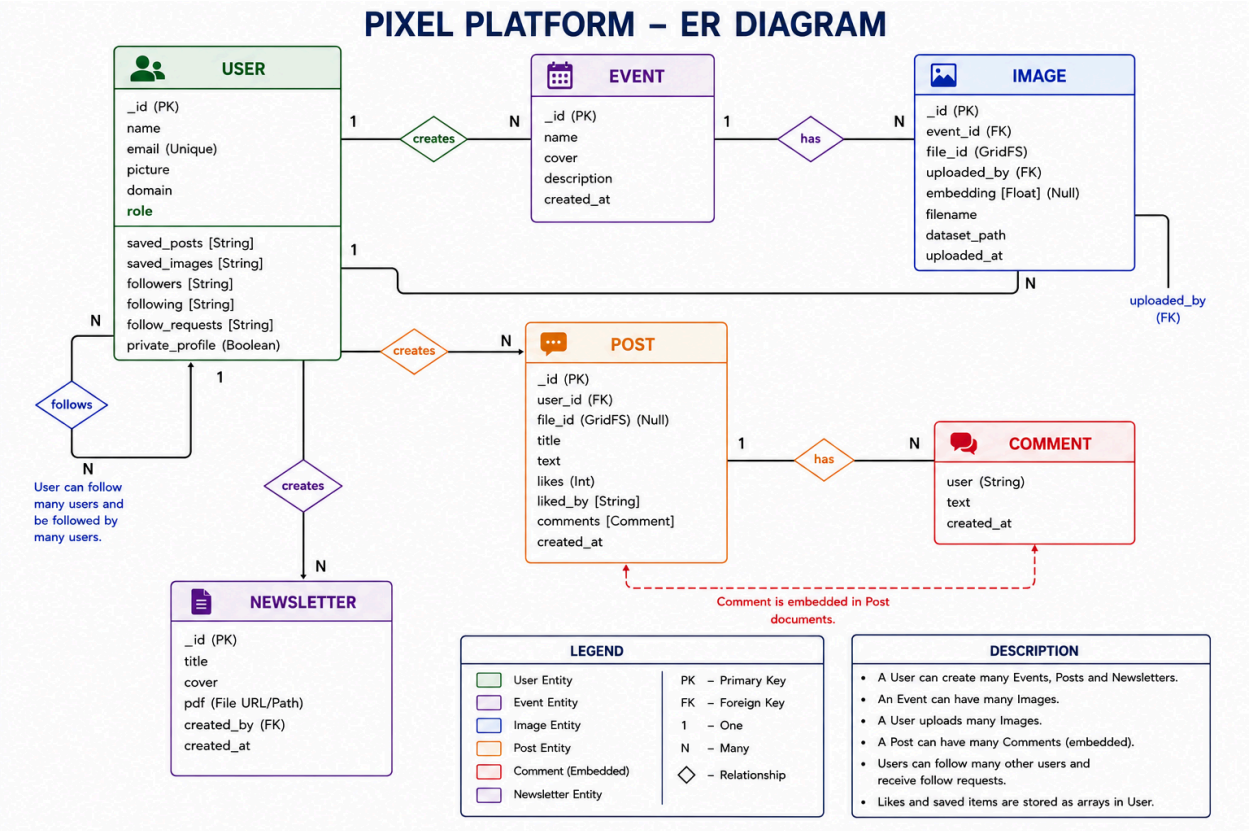

Figure 4.5: ER Diagram for Pixel System

4.8 Summary

The System Design chapter described the overall architecture and structural design of the PIXEL platform. It explained important design components such as the system flow diagram, data flow diagram, identified modules, and sequence diagram, which together represent the workflow and interaction between different parts of the system.

The chapter also provided a clear understanding of how users, administrators, backend services, database, and AI-based image search modules communicate with each other. These designs helped in creating a structured, scalable, and efficient system architecture for the successful implementation of the PIXEL platform.

CHAPTER 5

IMPLEMENTATION

5.1 Introduction to Implementation Phase

The Implementation Phase focuses on the actual development and integration of the PIXEL platform based on the system design and requirements analysis. In this phase, different modules such as user management, event handling, image processing, post management, newsletter services, and AI-based image search were developed and integrated into a complete working system.

The platform was implemented using modern technologies including React.js for frontend development, Django REST Framework and FastAPI for backend services, MongoDB for database management, and computer vision libraries for face recognition and image processing. The implementation phase ensured smooth communication between all modules and provided a scalable, responsive, and efficient platform for managing college events and AI-powered image retrieval.

5.2 Platform Used

5.2.1 Hardware Platform

The PIXEL platform was developed and tested on a standard computing system capable of handling web development, database management, and AI-based image processing tasks. The system used an Intel Core i5/i7 processor, minimum 8 GB RAM, and adequate SSD/HDD storage for storing datasets, images, newsletters, and project files. A stable internet connection was required for frontend-backend communication and cloud-based development support.

The hardware setup also included a webcam-enabled (optional) device for testing image uploads and AI-powered image search functionalities. The platform is designed to operate efficiently on standard desktop and laptop systems without requiring any specialized hardware configuration. The hardware environment provided sufficient performance for running React frontend services, Django and FastAPI backend servers, MongoDB database operations, and face recognition modules simultaneously.

5.2.2 Software Platform

The software platform used in the project is summarized below:

Component	Purpose
Visual Studio Code	Used as the primary code editor for project development.
React.js	Used for developing the responsive frontend user interface.
Django REST Framework	Used for building backend APIs and server-side functionalities.
FastAPI	Used for implementing the AI-based image search service.
MongoDB	Used for storing users, events, images, posts, and newsletter data.
Python	Used as the core programming language for backend and AI services.
JavaScript	Used for frontend development and client-side functionality.
Node.js	Used for managing frontend runtime and package dependencies.
face_recognition Library	Used for face detection and image matching functionalities.
GridFS	Used for storing and managing large image and media files in MongoDB.
Git and GitHub	Used for version control and project management.
Postman	Used for testing backend APIs and service endpoints.
HTML5, CSS3, and Tailwind CSS	Used for structuring, designing, and creating responsive user interfaces.
npm (Node Package Manager)	Used for installing and managing frontend packages and dependencies.

5.3 Overall Implementation Architecture

The PIXEL platform follows a modular full-stack architecture consisting of frontend, backend, database, and AI-based image search services. The frontend is developed using React.js, which provides an interactive and responsive user interface for browsing events, viewing galleries, accessing newsletters, exploring profiles, and interacting with platform features.

The backend is implemented using Django REST Framework for handling APIs, authentication, event management, image processing, post management, and communication with the database. MongoDB is used for storing users, events, posts, images, newsletters, and other application data, while GridFS manages large image and media files efficiently.

A separate AI-based image search service is developed using FastAPI, OpenCV, and face recognition libraries. This service processes uploaded images, generates facial embeddings, and retrieves matching photographs from the stored dataset.

Major Implementation Modules

- User Management and Authentication Module
- Event Management Module
- Image Upload and Processing Module
- Posts Upload and Processing Module
- Newsletter Module
- Face Recognition Service Module
- Database and Storage Management Module
- API Communication and Backend Services Module

5.4 User Management and Authentication Module

The User Management and Authentication Module is responsible for handling user registration, login, profile management, saved posts, saved images, and social interaction features such as follow requests and followers management. This module ensures secure access to the platform and enables users to interact with different features of the PIXEL platform.

Code Snippet:

```
from config.db import users_collection

def create_user(data):

email = data.get("email")

if not email:
return {"error": "Email required"}

existing = users_collection.find_one({"email": email})

if existing:
existing["_id"] = str(existing["_id"])
return existing

user = {
"name": "",
"email": email,
"picture": data.get("picture"),
"domain": data.get("domain"),
"role": data.get("role", ""),
"saved_posts": [],
"saved_images": []
}

result = users_collection.insert_one(user)

user["_id"] = str(result.inserted_id)

return user
```

5.4.1 Database Table Description

The Users collection stores all information related to registered users of the PIXEL platform. It contains user details such as name, email, profile picture, role, domain, saved posts, saved images, followers, following, follow requests, and profile privacy settings. This collection is mainly used for authentication, profile management, user interaction, and social connectivity features within the platform.

Figure 5.1: User Database Table

5.5 Event Management Module

The Event Management Module is used for creating and managing college events within the PIXEL platform. Administrators can add event details such as event name, cover image, and description. This module helps in organizing event galleries and related information systematically.

Code Snippet:

```
from config.db import events_collection
from datetime import datetime

def create_event(data):
event = {
"name": data.get("name"),
"cover": data.get("cover"),
"description": data.get("description"),
"created_at": datetime.utcnow()
}
```

```
result = events_collection.insert_one(event)

event["_id"] = str(result.inserted_id)

return event
```

5.5.1 Database Table Description

The Events collection stores information related to different college events such as technical fests, workshops, seminars, cultural activities, and sports events. It contains event details including event name, cover image, description, and creation date. This collection helps in organizing event galleries and displaying event-related content to users.


```
_id: ObjectId('69d258828d61590adccf193e')
name : "Cognoise 2K26"
cover : "69d258828d61590adccf1905"
description : "Annual Fest of SAIT "
created_at : 2026-04-05T12:41:38.436+00:00

_id: ObjectId('6a004eb5e6343470921bb762')
name : "TEDXSAIT"
cover : "6a004eb5e6343470921bb760"
description : "TEDX 2026"
created_at : 2026-05-10T09:24:05.494+00:00
```


Figure 5.2: Event Database Table

5.6 Image Upload and Processing Module

The Image Upload and Processing Module manages image uploads, image storage, and processing operations. It stores uploaded images using GridFS and generates image embeddings for AI-based image search functionalities. This module also connects uploaded images with event galleries.

Code Snippet:

```
from config.db import images_collection, fs


def add_image(file, data):

file_bytes = file.read()

file_id = fs.put(
file_bytes,
filename=file.name,
content_type=file.content_type
)

image = {
"event_id": data.get("event_id"),
"file_id": str(file_id),
"uploaded_by": data.get("uploaded_by")
}

result = images_collection.insert_one(image)

image["_id"] = str(result.inserted_id)

return image
```

5.6.1 Database Table Description

The Images collection manages uploaded event photographs and related metadata. It stores details such as event ID, uploaded user information, dataset path and AI-generated facial embeddings used for image matching. This collection plays an important role in image retrieval, gallery management, and AI-based image search functionalities.

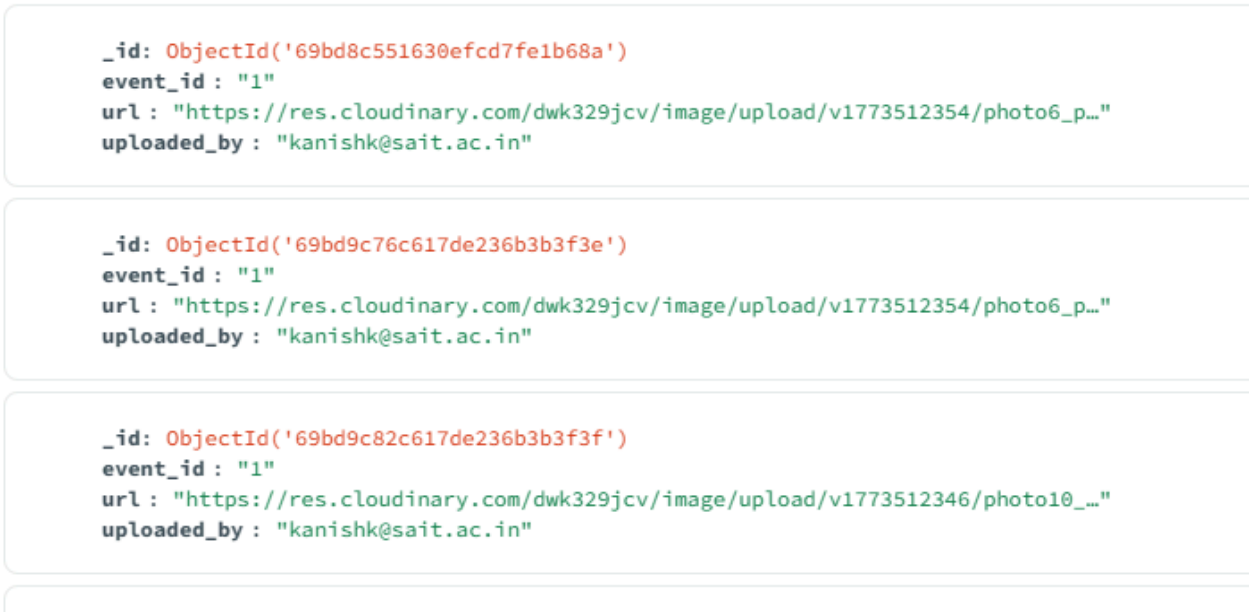

Figure 5.3: Image Database Table

5.7 Post Upload and Processing

The Posts Upload and Processing Module allows users and administrators to upload posts related to events, announcements, and activities. It also supports functionalities such as likes, comments, and post interactions to improve user engagement on the platform.

Code Snippet :

```
from config.db import posts_collection
from datetime import datetime

def create_post(file, data):

post = {
"title": data.get("title"),
"text": data.get("caption"),
"likes": 0,
"comments": [],
"created_at": datetime.utcnow()
}
result = posts_collection.insert_one(post)
```

```
post["_id"] = str(result.inserted_id)
return post
```

5.7.1 Database Table Description

The Posts collection stores posts related to announcements, updates, and campus activities. It contains information such as post title, caption, uploaded image references, likes count, liked users, comments, and creation date. This collection supports social interaction and user engagement features on the platform.

Figure 5.4: Post Database Table

5.8 Newsletter Module

The Newsletter Module is responsible for uploading and managing digital newsletters related to college events and activities. Users can access newsletters in PDF format directly through the platform.

Code Snippet:

```
from config.db import newsletters_collection
def create(data)
```

```
newsletter = {
"title": data.get("title"),
"cover": data.get("cover"),
"pdf": data.get("pdf")
}
result = newsletters_collection.insert_one(newsletter)
newsletter["_id"] = result.inserted_id
return newsletter
```

5.8.1 Database Table Description

The Newsletters collection stores digital newsletters associated with events and college activities. It contains newsletter titles, cover images, and PDF file references. This collection enables users to access and view newsletters directly through the platform interface.


```
_id: ObjectId('69bda1289c020d33cc672548')
title : "Pixel Newsletter – Delve 25:1"
cover : "/newsletters/delve_cover.png"
pdf : "/newsletters/Delve_1.pdf"
```

```
_id: ObjectId('69bda2089c020d33cc672549')
title : "Pixel Newsletter – Delve 25:1"
cover : "http://127.0.0.1:8000/static/newsletters/delve_cover.png"
pdf : "http://127.0.0.1:8000/static/newsletters/Delve_1.pdf"
```


Figure 5.5: Newsletter Database Table

5.9 Face Recognition and Processing Module

The Face Recognition Service Module provides AI-based image search functionality using computer vision and face recognition techniques. The module processes uploaded images, generates facial embeddings, and compares them with stored images to retrieve matching photographs.

Code Snippet:

```
from fastapi import FastAPI, UploadFile, File, Form
import shutil
import os
```

```
import threading

from backend.dataset_watcher import start_watcher
from backend.search_engine import search_faces
from utils.config import UPLOAD_DIR

app = FastAPI()

os.makedirs(UPLOAD_DIR, exist_ok=True)

# ✅ KEEP SIMPLE ROUTE + ADD event_id FROM FORM
@app.post("/search")
async def search_face(
    file: UploadFile = File(...),
    event_id: str = Form(...)
):
    try:
        path = os.path.join(UPLOAD_DIR, file.filename)
        with open(path, "wb") as buffer:
            shutil.copyfileobj(file.file, buffer)
        print("🔍 Searching for event:", event_id)
        # ✅ FIX: pass event_id
        matches = search_faces(path, event_id)
        return {"matches": matches}
    except Exception as e:
        print("❌ SEARCH ERROR:", e)
        return {"matches": []}
@app.on_event("startup")
def start_background_tasks():
    threading.Thread(target=start_watcher, daemon=True).start()
```

5.9.1 Database Table Description

GridFS is a file storage system provided by MongoDB for storing large files such as images, PDFs, and media content efficiently. It stores files in smaller chunks within MongoDB to improve storage and retrieval performance.

In the PIXEL platform, GridFS is used to store event images, post media, and newsletters. These stored images are accessed by the face recognition service to generate facial embeddings and perform AI-based image matching. GridFS helps in efficient image handling, smooth database communication, and scalable support for the face recognition module.

Figure 5.6: FS chunks Database Table

5.10 Module Integration

Module Integration is the process of connecting different modules of the PIXEL platform to work together as a complete and functional system. After the individual development of frontend components, backend services, database operations, and AI-based image search functionalities, all modules were integrated to ensure smooth communication and data flow between them.

The React.js frontend communicates with the Django REST Framework backend through APIs for handling user requests, event management, image uploads, posts, and newsletters. MongoDB and GridFS are integrated with the backend for efficient data and media storage. The FastAPI-based face recognition service is connected separately to process uploaded images and perform AI-based image search operations.

The integration process ensured proper interaction between all modules, reliable API communication, efficient database connectivity, and smooth execution of system functionalities across the entire PIXEL platform.

5.11 Implementation Summary

The Implementation Phase involved the development and integration of all major modules of the PIXEL platform, including user management, event handling, image processing, posts, newsletters, and AI-based image search. The system was developed using React.js, Django REST Framework, FastAPI, MongoDB, and computer vision technologies.

All frontend, backend, database, and AI service modules were successfully integrated to ensure smooth communication, efficient data handling, and reliable system performance across the platform.

5.11 Testing

5.11.1 Introduction to Testing

Testing is an important part of the implementation phase, especially in full-stack applications involving frontend, backend, database, and AI-based services. In the PIXEL platform, testing is performed during and after implementation to ensure that all modules function correctly before complete system integration.

Since the project is developed at the academic level, advanced industrial testing tools are not used. Instead, suitable manual, functional, and integration testing techniques are applied to verify the working of different platform features and services.

5.11.2 Testing Objectives

The primary objectives of testing during implementation are:

- To verify correct functioning of frontend and backend modules
- To ensure proper event browsing and image management
- To validate AI-based image search functionality
- To check smooth communication between APIs and database services
- To ensure proper handling of posts, newsletters, and user profiles
- To verify overall system responsiveness and stability

5.11.3 Testing Techniques Used

Functional Testing

Functional testing is used to verify whether each feature of the PIXEL platform performs according to its intended functionality. Different modules such as user authentication, event management, image uploads, AI-based image search, posts handling, newsletters, and profile management are tested individually to ensure correct execution of operations and proper system responses.

Manual Testing

Manual testing is performed by directly interacting with the platform through real user actions. Features such as registration, login, event browsing, image uploads, AI image search,

profile viewing, post interactions, and newsletter access are tested manually to verify usability, responsiveness, and overall user experience. This method helps in identifying interface issues and validating real-time system behavior.

Integration Testing

Integration testing is performed to ensure smooth communication and data exchange between different modules of the platform. The React.js frontend, Django REST Framework backend, MongoDB database, GridFS storage, and FastAPI-based image search service are tested together to verify proper API communication, database connectivity, image processing, and response handling across the complete system.

System Testing

System testing is conducted after integrating all modules to evaluate the PIXEL platform as a complete working system. The testing verifies overall system performance, stability, reliability, and workflow execution under normal operating conditions. Different functionalities are tested together to ensure the platform operates efficiently as a unified full-stack application.

5.11.4 Test Cases Used During Implementation

Test Case ID	Functionality Tested	Expected Outcome
TC01	User Registration/Login	User authentication works successfully
TC02	Event Browsing	Events and galleries load correctly
TC03	Image Upload	Images are uploaded and stored successfully
TC04	AI Image Search	Matching photographs are retrieved correctly
TC05	Post Upload	Posts are uploaded and displayed properly
TC06	Newsletter Viewing	PDF newsletters open successfully
TC07	Student Profile Access	User profiles are displayed correctly

TC08	API Communication	Frontend and backend communicate properly
TC09	Database Storage	Data is stored and retrieved successfully
TC10	System Logout	User session ends securely

All test cases were executed during implementation and produced the expected results under normal operating conditions.

5.11.5 Test Environment

Testing was performed in the same environment used for implementation:

- Operating System: Windows
- Frontend Framework: React.js
- Backend Framework: Django REST Framework and FastAPI
- Database: MongoDB
- Programming Language: Python and JavaScript
- Libraries: face_recognition
- Development Tools: Visual Studio Code and Postman

Testing was performed under normal system and network conditions to maintain consistent platform performance.

5.11.6 Testing Limitations

Due to the academic scope of the project, the following limitations apply:

- Automated testing tools are not used.
- Stress and load testing are not performed.
- AI image search accuracy may vary depending on image quality, lighting conditions, and facial visibility.
- Performance evaluation is based mainly on manual testing and observation.
- Testing is conducted on a limited number of devices and browsers.
- Large-scale real-world dataset testing is not performed.
- Network performance and internet speed may affect API response time during testing.

- Security and penetration testing are not performed extensively.
- Simultaneous multi-user testing is limited during implementation.
- Testing is performed under normal operating conditions and may vary under extreme system loads.

5.11.7 Testing Summary

Testing performed during the implementation phase confirmed that the PIXEL platform operates correctly and satisfies the defined functional requirements. Functional, manual, integration, and system testing techniques ensured reliable communication between all modules and stable execution of platform functionalities.

CHAPTER 6

CONCLUSION

6.1 Introduction

The Conclusion chapter summarizes the overall development, implementation, and outcomes of the PIXEL platform. It highlights how the project successfully integrates modern web technologies, database management, and AI-based image search functionalities to create a smart event gallery and student interaction platform.

This chapter discusses the important features achieved during development, the limitations of the current system, and the possible future enhancements that can further improve the platform. It also reflects the practical application of full-stack development and computer vision technologies in solving real-world event management and image retrieval problems.

6.2 Important Features of the System

The PIXEL system offers several important features that enhance usability and demonstrate practical applicability:

- AI-based image search using face recognition techniques.
- Smart event gallery management for college events and activities.
- User registration, authentication, and profile management.
- Event browsing and interactive gallery viewing.
- Image upload and processing using GridFS storage.
- Posts upload, likes, comments, and social interaction features.
- Student profile exploration and connection system.
- Newsletter management and PDF viewing support.
- Responsive and user-friendly frontend interface.
- Separate FastAPI-based face recognition service for image processing.
- Efficient database management using MongoDB.
- Modular full-stack architecture for scalability and maintainability.
- API-based communication between frontend, backend, database, and AI services.
- Secure and organized media storage and retrieval system.

These features collectively provide a seamless and interactive user experience without the need for additional hardware.

6.3 Limitations of the System

Despite its successful implementation, the PIXEL system has certain limitations, mainly due to its academic scope and dependency on environmental conditions:

- AI-based image search accuracy may vary depending on image quality, lighting conditions, and facial visibility.
- Processing and matching large numbers of images may increase image retrieval time.
- The efficiency of face recognition depends on properly sorted and organized event image datasets.
- Real-time processing performance may reduce when handling very large image collections simultaneously.
- The system currently supports limited large-scale multi-user testing.
- Internet speed and network conditions may affect API response time and overall system performance.
- Automated testing and advanced security testing are not fully implemented.
- The platform is mainly optimized for standard desktop and laptop environments.
- Performance may vary on low-configuration systems with limited processing power and memory.

These limitations highlight areas where further improvements can be made.

6.4 Future Work

The PIXEL system has significant potential for future enhancements and expansion. Possible future work includes:

- Development of a dedicated Android mobile application for improved accessibility and user experience.
- Integration of real-time chat and messaging features for communication between students and users.
- Support for video uploads and video-based posts in addition to image content.
- Expansion of the platform into a complete college social media and community interaction system.
- Implementation of real-time notifications and activity updates for events, posts, and user interactions.

- Enhancement of AI-based image search with improved accuracy and faster processing techniques.
- Addition of advanced recommendation systems for events, posts, and student connections.
- Cloud deployment and scalable distributed storage for handling larger datasets and increasing users.
- Integration of real-time live event streaming and media sharing features.
- Development of advanced analytics dashboards for event insights and user engagement monitoring.
- Support for multi-face recognition and automatic grouping of event photographs.
- Implementation of stronger security, privacy controls, and advanced authentication mechanisms.
- Addition of role-based access systems for students, clubs, and administrators.
- Integration of AI-powered content moderation and smart media organization features.

These enhancements can significantly improve the system's usability, scalability, and real-world applicability.

6.5 Summary

The Conclusion chapter summarized the development, implementation, features, limitations, and future scope of the PIXEL platform. The project successfully integrated frontend, backend, database, and AI-based image search functionalities into a smart event gallery and student interaction platform.

The chapter also highlighted the practical use of modern web technologies and artificial intelligence in improving event management, image retrieval, and digital student interaction systems.

CHAPTER 7

REFERENCES

REFERENCES

[1] React.js Documentation, "React – A JavaScript library for building user interfaces," Meta Platforms, Inc. [Online]. Available: https://react.dev/

[2] Django REST Framework Documentation, "Django REST Framework," Encode OSS Ltd. [Online]. Available: https://www.django-rest-framework.org/

[3] FastAPI Documentation, "FastAPI Framework," Sebastián Ramírez. [Online]. Available: https://fastapi.tiangolo.com/

[4] MongoDB Documentation, "MongoDB Manual," MongoDB Inc. [Online]. Available: https://www.mongodb.com/docs/

[5] face_recognition Library Documentation, "Face Recognition Library," Adam Geitgey. [Online]. Available: https://face-recognition.readthedocs.io/

[6] GridFS Documentation, "MongoDB GridFS," MongoDB Inc. [Online]. Available: https://www.mongodb.com/docs/manual/core/gridfs/

[7] Node.js Documentation, "Node.js JavaScript Runtime," OpenJS Foundation. [Online]. Available: https://nodejs.org/

[8] Tailwind CSS Documentation, "Tailwind CSS Framework," Tailwind Labs. [Online]. Available: https://tailwindcss.com/

[9] GitHub Documentation, "GitHub Developer Platform," GitHub Inc. [Online]. Available: https://docs.github.com/

[10] Postman Documentation, "Postman API Platform," Postman Inc. [Online]. Available: https://www.postman.com/

[11] Python Documentation, “Python Programming Language,” Python Software Foundation. [Online]. Available: https://docs.python.org/3/

[12] Google Photos, “Google Photos – Photo Storage and Face Grouping System,” Google LLC. [Online]. Available: https://photos.google.com/

[13] Flickr, “Flickr – Image Sharing and Media Management Platform,” SmugMug, Inc. [Online]. Available: https://www.flickr.com/

[14] Instagram, “Instagram – Social Networking and Media Sharing Platform,” Meta Platforms, Inc. [Online]. Available: https://www.instagram.com/

[15] PyMongo Documentation, “PyMongo MongoDB Driver,” MongoDB Inc. [Online]. Available: https://pymongo.readthedocs.io/

[16] Uvicorn Documentation, “Uvicorn ASGI Server,” Encode OSS Ltd. [Online]. Available: https://www.uvicorn.org/

[17] Visual Studio Code Documentation, “Visual Studio Code,” Microsoft Corporation. [Online]. Available: https://code.visualstudio.com/

[18] NumPy Documentation, “NumPy Scientific Computing Library,” NumPy Developers. [Online]. Available: https://numpy.org/

[19] MDN Web Docs, “HTML, CSS and JavaScript Documentation,” Mozilla Foundation. [Online]. Available: https://developer.mozilla.org/

Reference Notes

- All references listed in this report were used for understanding, designing, developing, and implementing different modules of the PIXEL platform.
- Official documentation of frameworks, libraries, and development tools was referred to during frontend, backend, database, and AI service implementation.

- Online platforms such as Google Photos, Flickr, and Instagram were studied for understanding image management, social interaction, and media sharing concepts.
- Research materials, developer communities, and technical documentation were used for resolving implementation challenges and improving system functionality.
- All referenced resources belong to their respective organizations and authors.